

fractional knapsack

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Item
```

```
{
```

```
    float ratio;
```

```
    int value, weight;
```

```
};
```

```
void swap (struct Item a[], int i, int j)
```

```
{
```

```
    struct Item temp = a[i];
```

```
    a[i] = a[j];
```

```
    a[j] = temp;
```

```
}
```

```
void sort (struct Item a[], int n)
```

```
{
```

```
    for (int i = 0; i < n - 1; i++)
```

```
        for (int j = 0; j < n - 1 - i; j++)
```

```
        {
```

```
            if (a[j].ratio < a[j+1].ratio)
```

```
            {
```

```
                swap(a, j, j+1);
```

```
            }
```

```
        }
```

```
}
```

```
float fractionalKnapsack (struct Item a[], int n, int capacity)
```

```
{
```

```
    float value = 0.0;
```

```

for (int i=0; i<n; i++)
{
    if (capacity >= a[i].weight)
    {
        totalvalue += a[i].value;
        capacity -= a[i].weight;
    }
    else
    {
        value += a[i].ratio * capacity;
        break;
    }
}
return value;
}

```

```

int main()
{

```

```

    int n, capacity;

```

```

    printf("Enter number of item: ");

```

```

    scanf("%d", &n);

```

```

    struct item a[n];

```

```

    for (int i=0; i<n; i++)
    {

```

```

        printf("Enter the value & weight of item %d: ", i+1);

```

```

        scanf("%d %d", &a[i].value, &a[i].weight);

```

```

        a[i].ratio = (float) a[i].value / a[i].weight;
    }
}

```

```

printf ("Enter knapsack capacity: ");
scanf ("%d", &capacity);
sort (a, n);
clock_t start, end; start = clock();
float maxVal = fracknap (a, n, capacity);
end = clock();

```

```

printf ("Maximum profit = %.2f\n",
        maxVal);

```

```

double time = (double)(end - start) /
               CLOCKS_PER_SEC;

```

```

printf ("Execution time: %.f\n", time);

```

```

return 0;

```

~~Q13/4~~ Output:

Enter number of items: 3

Enter value & weight of item 1: 25 18

2: 24 15

3: 15 10

Enter knapsack capacity: 20.

Maximum profit = 31.50

Execution time: 0.00.